Universidade Estadual de Santa Cruz (UESC)

Etapa 2 — Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time): Implementação sequencial e paralela em memória distribuída com MPI

Relatório acadêmico — DEC107 — Processamento Paralelo

Autores: Gabriel Oliveira Lucas César — Matrícula: 202410123;
Ronaldo Ribeiro Porto Filho — Matrícula: 202410131

Disciplina: DEC107 — Processamento Paralelo
Data: 29/10/2026

---

1 Introdução

Este relatório apresenta a Etapa 2 do projeto de geração do conjunto de Mandelbrot pelo método *escape-time*, agora com processos MPI e memória distribuída. A proposta é comparar uma implementação MPI independente com o código sequencial original da Etapa 1, estudar duas formas estáticas de decomposição da imagem e medir o efeito do número de processos, da granularidade das tarefas e da coleta dos resultados sobre o desempenho.

Para cada pixel, um número complexo c é determinado por interpolação linear das coordenadas da imagem. A partir de $z_0 = 0$, aplica-se

$$z_{n+1} = z_n^2 + c, \qquad \text{escape quando } |z_{n+1}|^2 > 4, \tag{1}$$

até o limite `MAX_ITER`. O número de iterações executadas é salvo numa matriz de inteiros de 32 bits. Embora os pixels sejam independentes, o esforço computacional é heterogêneo: muitos escapam rapidamente e outros exigem centenas ou milhares de iterações. Consequentemente, distribuir quantidades iguais de linhas pode não distribuir quantidades iguais de trabalho.

Diferentemente do OpenMP, no qual *threads* compartilham o mesmo espaço de endereçamento, os processos MPI possuem buffers próprios e precisam trocar dados explicitamente. O particionamento define o trabalho de cada processo, e a coleta da matriz final integra o tempo medido. O envio dos parâmetros e a alocação dos buffers ocorrem antes da janela cronometrada. Foram implementadas *faixas contíguas* e *distribuição cíclica de blocos de linhas*; ambas usam atribuição **estática**. Em um esquema dinâmico mestre–trabalhador, tarefas seriam entregues conforme os processos terminassem o trabalho, com potencial de melhor adaptação ao custo irregular e maior custo de coordenação; esse esquema não foi implementado nesta etapa.

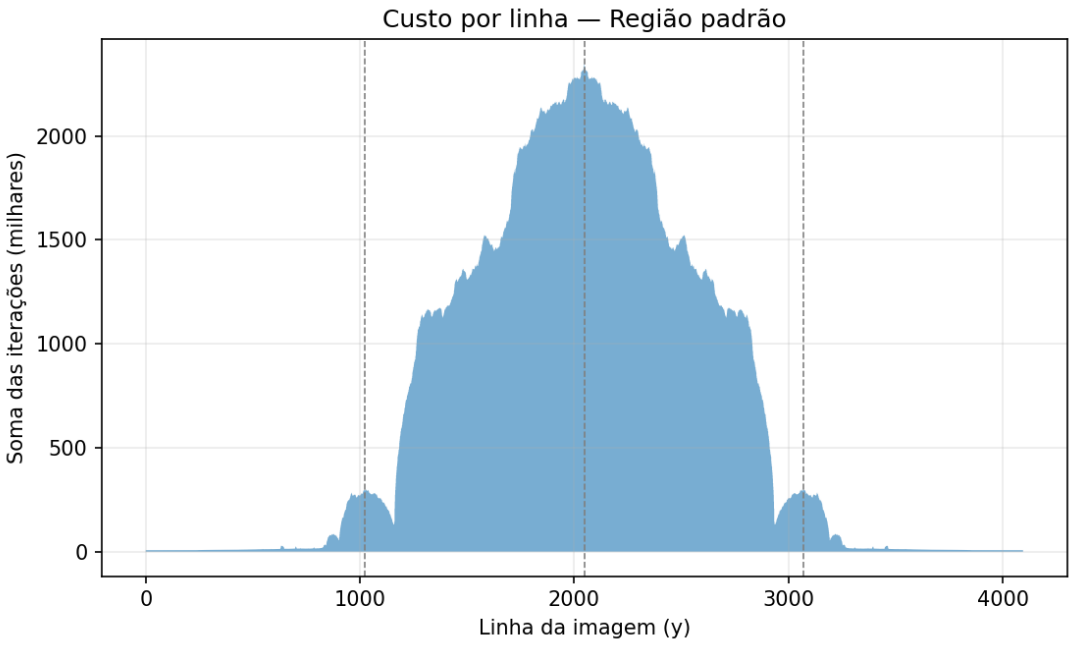


Figura 1: Trabalho computacional agregado por linha na região padrão; sua concentração no centro explica o risco de desbalanceamento por faixas contíguas.

2 Metodologia

2.1 Implementações e particionamento

O código `mandelbrot_sequencial.c` da Etapa 1 foi preservado sem alterações. O programa `mandelbrot_mpi.c` implementa o mesmo núcleo *escape-time* com precisão `double`, escrita de matriz `int32` em ordem de linhas e escolha da distribuição por parâmetros de execução. Cada processo computa apenas suas linhas atribuídas, sem compartilhamento de endereços.

Blocos contíguos: as H linhas são divididas em p faixas consecutivas, diferindo no máximo uma linha em quantidade. O *rank* r calcula sua faixa, mas pode receber uma região mais custosa do fractal.

Cíclica com *chunk*: blocos consecutivos de c linhas são atribuídos alternadamente entre os processos pela regra

$$\text{rank}(y) = \left\lfloor \frac{y}{c} \right\rfloor \bmod p, \qquad c \in \{1, 4, 16\}. \tag{2}$$

A estratégia espalha partes distantes da imagem entre os processos. O particionamento é definido previamente: não há fila dinâmica de tarefas, nem entrega de trabalho sob demanda.

2.2 Operações MPI e composição da imagem

O programa utiliza `MPI_Init`, `MPI_Comm_rank` e `MPI_Comm_size` para inicialização e identificação dos processos; `MPI_Bcast` para distribuição de parâmetros; `MPI_Allreduce` para detecção coletiva de falhas de alocação; e `MPI_Barrier` para alinhamento do início e encerramento da janela medida. Cada processo produz uma parte da matriz em buffer próprio.

A coleta ocorre por `MPI_Gatherv`, adequado porque a quantidade de linhas por processo pode variar. Para faixas contíguas, os dados são recebidos diretamente na ordem da imagem. Na estratégia

cíclica, o processo raiz recebe os buffers agrupados por *rank* e depois reorganiza as linhas nas posições globais, com tempo de montagem medido separadamente. `MPI_Gather` reúne métricas locais e contagens de trabalho, enquanto `MPI_Reduce` obtém a maior duração local. A saída binária é escrita somente pelo processo raiz, após a janela principal de cálculo e coleta.

Não se implementou uma terceira estratégia dinâmica nem um mestre dedicado exclusivamente ao despacho: o *rank* zero também calcula pixels. Assim, a "sobrecarga do mestre" corresponde aqui a coleta, reorganização da matriz quando cíclica, agregação de métricas e escrita, não a gerenciamento dinâmico de tarefas.

2.3 Casos, ambiente e protocolo experimental

Foram avaliados dois casos de escalabilidade forte, ambos em 4096×4096: a região padrão ($\mathrm{Re} \in [-2,1]$, $\mathrm{Im} \in [-1{,}5,1{,}5]$, `MAX_ITER=1000`) e o zoom no vale dos cavalos-marinhos (centro $-0{,}743643887 + 0{,}131825904i$, largura 3×10^{-3} e `MAX_ITER=5000`). Para ambos, foram usados $p \in \{1,2,4,6,8,12\}$ processos, nas quatro configurações: blocos, cíclica com *chunk* 1, 4 e 16.

A escalabilidade fraca foi investigada na região padrão, com resolução aproximada $4096\sqrt{p}$ em cada dimensão: 4096^2, 5793^2, 8192^2, 10033^2, 11585^2 e 14189^2, respectivamente para $p = 1,2,4,6,8,12$. Isso mantém aproximadamente 4096^2 pixels por processo, mas não garante o mesmo número de iterações por processo.

Tabela 1: Ambiente de execução da Etapa 2.

Item	Configuração
CPU	Intel Core i5-12400F, 6 núcleos físicos, 12 processadores lógicos
Memória	32 GB de RAM
Sistema	Windows 11
Sequencial	GCC/MinGW 6.3.0 (32 bits), `-O2 -fopenmp -ffp-contract=off -msse2 -mfpmath=sse`
MPI	Microsoft Visual C++ x64 (MSVC), `/O2 /TC /fp:precise`, MS-MPI `10.1.12498.18`; MSVC `195136260` (registro do executável)
Execução MPI	`mpiexec -n N` em um único computador
Condições manuais	Plano de energia, temperatura, aplicativos abertos e estado do OneDrive não foram registrados nesta coleta. O comando não solicita binding explícito; a afinidade efetiva não foi registrada.

A bateria oficial totalizou **386 execuções com verificação interna aprovada**: 360 medições MPI (72 configurações, com 5 repetições cada), 20 referências sequenciais fortes (7 no início e 3 no fim de cada região) e 6 referências sequenciais fracas (uma por resolução). Os registros são de 28/10/2026, no computador de Gabriel. A validade interna dessas execuções não implica aprovação no critério externo de corretude. As configurações MPI foram embaralhadas dentro de cada rodada; o resultado consolidado foi resumido pela **mediana**, com mínimo, máximo e desvio-padrão preservados nos CSVs. A bateria grava a matriz binária, sem solicitar PPM; a escrita fica fora do tempo principal. Para a inspeção visual exigida no enunciado, foram geradas posteriormente imagens PGM das matrizes preservadas de 4096^2, mantendo suas contagens, sem nova medição de desempenho. A resolução declarada de `MPI_Wtime`, obtida por `MPI_Wtick`, foi de 10^{-7} s. Isso descreve a resolução do relógio, não a precisão estatística das medições.

A medição MPI inicia após uma barreira e abrange cálculo, `MPI_Gatherv`, reorganização no raiz quando aplicável e barreira final. O total usado na análise é o máximo das durações locais, reduzido

com `MPI_MAX`; não são subtraídos relógios de processos diferentes. Inicialização de MPI, criação de processos, broadcast de parâmetros, alocação, barreira inicial, agregação das métricas e escrita estão fora dessa janela. O cálculo local e a duração local da coleta são medidos com `MPI_Wtime`; o código sequencial preserva o cronômetro `omp_get_wtime` da Etapa 1. A duração de `MPI_Gatherv` **inclui espera/sincronização** e não pode ser interpretada diretamente como tempo de transferência na rede. Os testes foram locais, com MPI entre processos na mesma máquina.

2.4 Métricas

O tempo de referência T_{seq} é a mediana sequencial correspondente a cada caso e resolução (na escalabilidade fraca, existe apenas uma medição sequencial por tamanho). Para a escalabilidade forte:

$$S(p) = \frac{T_{seq}}{T_{MPI}(p)}, \qquad E(p) = \frac{S(p)}{p} \times 100\%. \tag{3}$$

O fator de balanceamento de carga (FBC), tal como definido no modelo da disciplina, utiliza **somente o tempo de computação de cada processo**, excluindo comunicação e escrita:

$$\mathrm{FBC} = \frac{T^{calc}_{\max} - T^{calc}_{\min}}{T^{calc}_{\max}}, \quad 0 \le \mathrm{FBC} \le 1. \tag{4}$$

Valores próximos de zero indicam tempos de cálculo parecidos; próximos de um indicam grande disparidade. Para escalabilidade fraca foi adotada $E_{fraca}(p) = 100\, T_{MPI}(1, N_1)/T_{MPI}(p, N_p)$, comparando a **mesma estratégia de distribuição** em diferentes resoluções. Ela é uma eficiência temporal relativa, não a eficiência forte S/p.

3 Resultados

3.1 Corretude

As **386 execuções oficiais** apresentaram verificações internas de `hash_ok` e `config_ok` válidas. Os hashes registrados nas 360 execuções MPI coincidem com a referência sequencial do mesmo caso e resolução. Os dez binários preservados de 4096^2 (sequencial e quatro configurações MPI, nas duas regiões) também tiveram seus hashes reconferidos. Os binários maiores da escalabilidade fraca não foram todos preservados pelo script, que limita a cópia de referências a 100 MiB; nesses casos, a evidência disponível é o hash registrado durante a execução. Na comparação explícita das matrizes preservadas de 4096^2, tanto blocos quanto cíclica resultaram em **zero pixels diferentes** em relação à versão sequencial.

A comparação *externa* com a função Python do professor **não foi aprovada integralmente**. A referência Python registra índice de escape i, enquanto o C registra $i + 1$; essa convenção foi ajustada apenas na comparação diagnóstica, sem alterar arquivos ou medições. Mesmo após o ajuste, restaram 786 pixels divergentes (0,004685%, diferença máxima 460) no padrão e 24.242 (0,144494%, diferença máxima 1.415) no zoom. Os limites da Seção 5.5 do enunciado são 0,01% e diferença máxima de uma iteração; portanto, as duas regiões ficaram fora dos critérios completos. A divergência também está presente no sequencial preservado da Etapa 1 e **não foi introduzida pela distribuição MPI**. Sem o ajuste diagnóstico, divergem 83,225721% dos pixels no padrão (diferença máxima 461) e 99,974447% no zoom (máxima 1.414). Portanto, os tempos permanecem resultados experimentais da implementação entregue, mas **não demonstram elegibilidade à pontuação de desempenho**: o enunciado condiciona essa pontuação ao critério externo de corretude.

Tabela 2: Corretude: comparação diagnóstica após ajuste de convenção de contagem.

Caso	Pixels divergentes	Percentual	Diferença máxima
Padrão	786	0,004685%	460
Zoom	24.242	0,144494%	1.415

3.2 Escalabilidade forte: tempo, speedup e eficiência

Para 4096^2, a referência sequencial mediana foi de **6,9825 s** na região padrão e **3,0890 s** no zoom. A Tabela 3 apresenta as medianas das quatro configurações para cada número de processos. Os melhores tempos medianos em 12 processos foram obtidos com *chunk* 16 no padrão e *chunk* 4 no zoom; o melhor *chunk* não é necessariamente o mesmo nos demais pontos.

Tabela 3: Tempos medianos MPI (s), 4096^2; cinco repetições por célula.

Caso	p	Blocos	Cíclica $c = 1$	Cíclica $c = 4$	Cíclica $c = 16$
Padrão	1	7,278	7,225	7,203	7,362
Padrão	2	3,784	3,808	3,806	3,822
Padrão	4	3,667	2,009	2,004	2,010
Padrão	6	3,141	1,401	1,393	1,398
Padrão	8	2,641	1,075	1,074	1,084
Padrão	12	1,988	0,830	0,829	0,804
Zoom	1	3,263	3,286	3,288	3,269
Zoom	2	1,991	1,740	1,748	1,730
Zoom	4	1,093	0,942	0,929	0,932
Zoom	6	0,828	0,650	0,644	0,650
Zoom	8	0,607	0,510	0,503	0,503
Zoom	12	0,489	0,476	0,430	0,438

Na região padrão, com 12 processos e *chunk* 16, o MPI alcançou **speedup 8,682** e **eficiência 72,35%**; por blocos, obteve speedup 3,512 e eficiência 29,26%. No zoom, com 12 processos e *chunk* 4, obteve speedup **7,183** e eficiência **59,86%**; por blocos, speedup 6,318 e eficiência 52,65%. Esses valores usam a **referência sequencial medida nesta bateria da Etapa 2**, não os tempos históricos da Etapa 1 nem $T_{MPI}(1)$. Por exemplo, $6{,}9825/0{,}804258 \simeq 8{,}682$ e $100 \times 8{,}682/12 \simeq 72{,}35\%$. O sequencial foi compilado com GCC de 32 bits e o MPI com MSVC de 64 bits: o speedup compara essas implementações completas e não isola apenas o efeito do paralelismo. Em $p = 1$, o MPI levou cerca de 3,2–5,4% a mais que o sequencial no padrão e 5,6–6,4% a mais no zoom.

A mediana sequencial inicial/final passou de 7,066 para 6,831 s no padrão (deriva de $-3{,}33\%$) e de 3,085 para 3,115 s no zoom ($+0{,}97\%$). No zoom com 12 processos, o coeficiente de variação chegou a 11,60% em cíclica $c = 1$ e 11,41% em blocos. As faixas mínimo–máximo se sobrepõem entre os *chunks*; cinco repetições não sustentam afirmar superioridade estatística universal de $c = 16$ ou $c = 4$. As barras dos gráficos mostram variação observada, não intervalos de confiança.

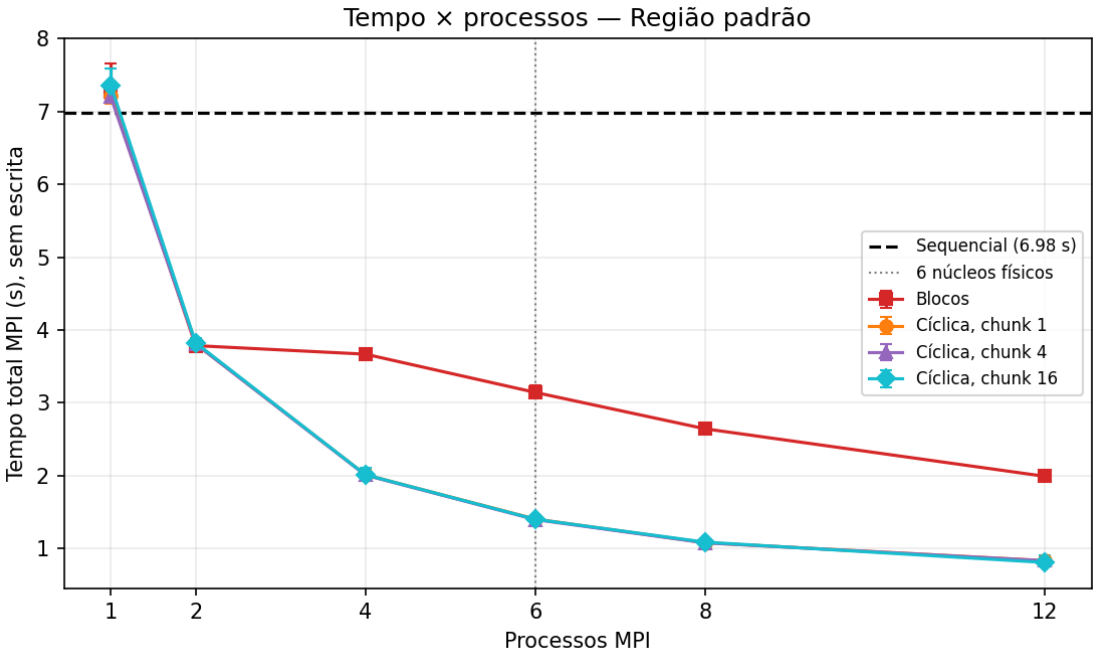


Figura 2: Escalabilidade forte: tempo mediano MPI na região padrão, com barra de variação entre mínimo e máximo e referência sequencial.

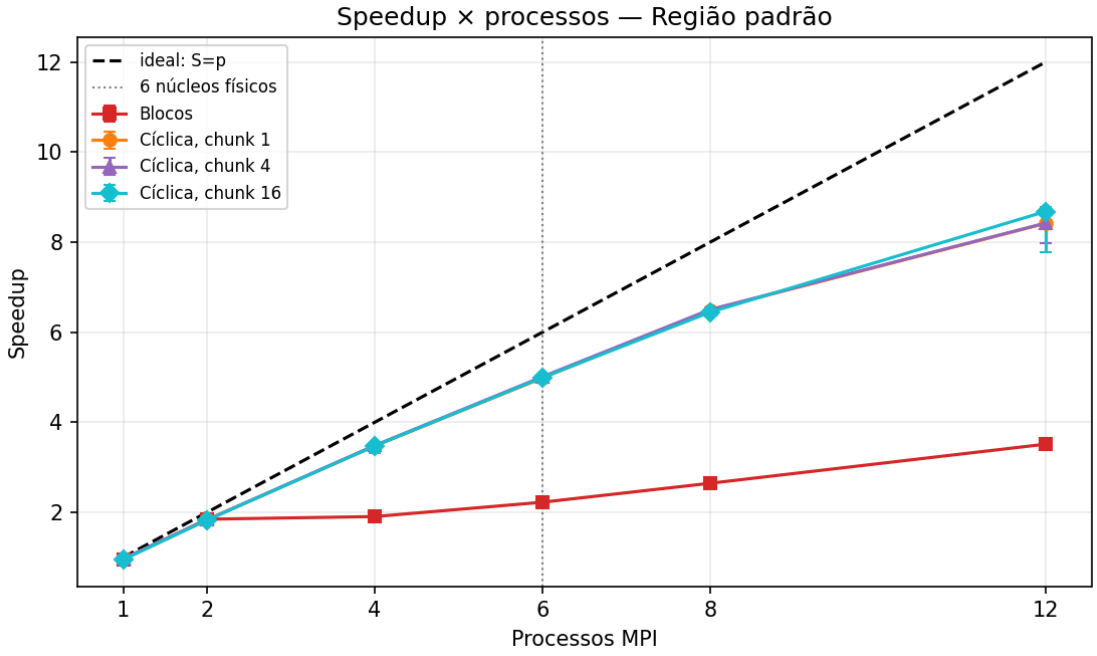


Figura 3: Speedup na região padrão por estratégia, com linha ideal $S(p) = p$.

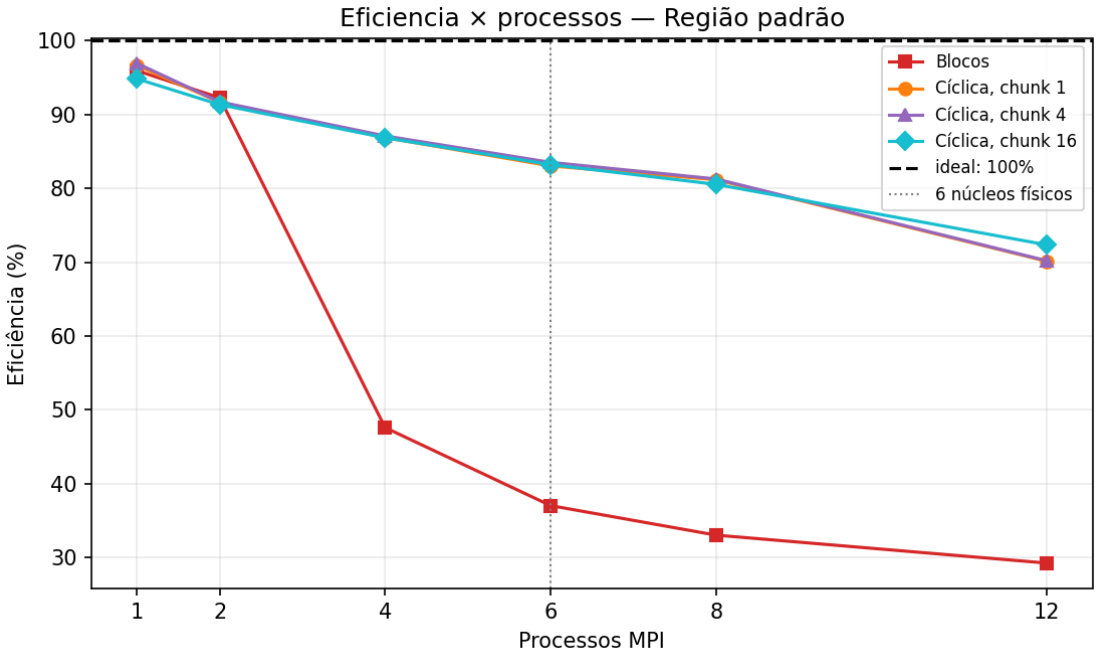


Figura 4: Eficiência paralela na região padrão; a linha vertical destaca os seis núcleos físicos.

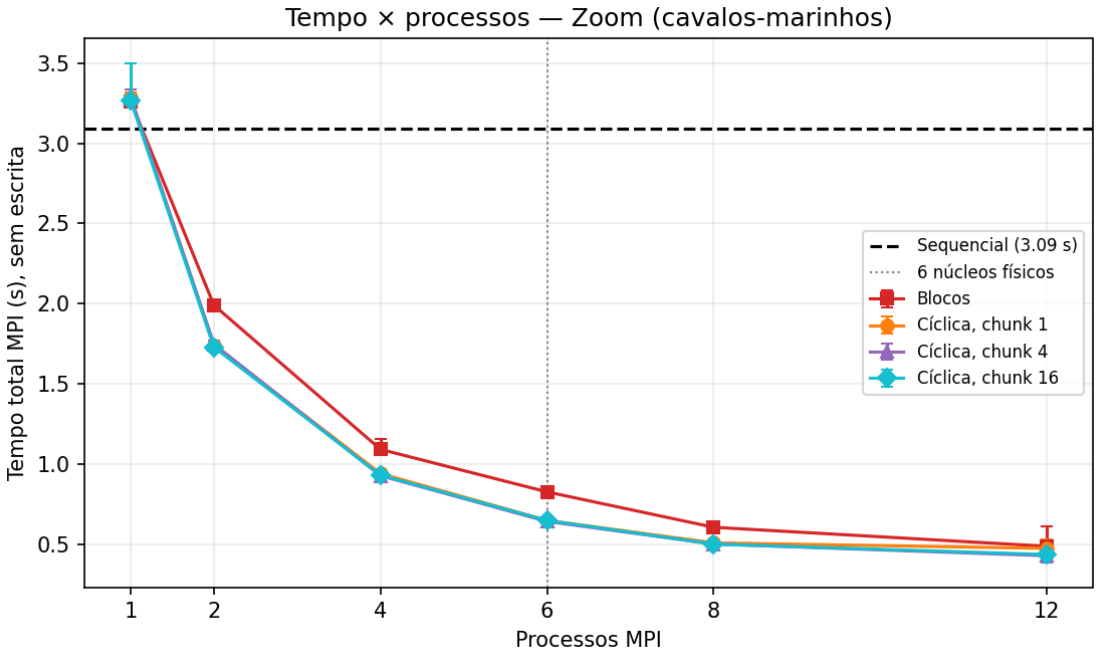


Figura 5: Tempo mediano no zoom, com variação entre mínimo e máximo das cinco repetições e referência sequencial.

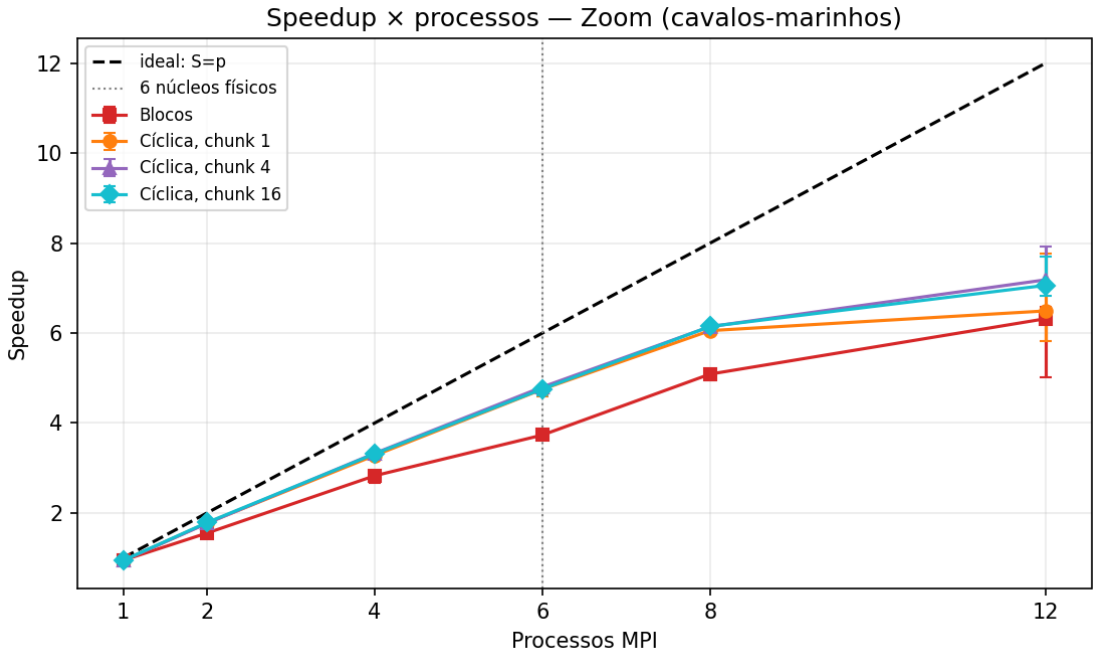


Figura 6: Speedup no zoom do vale dos cavalos-marinhos.

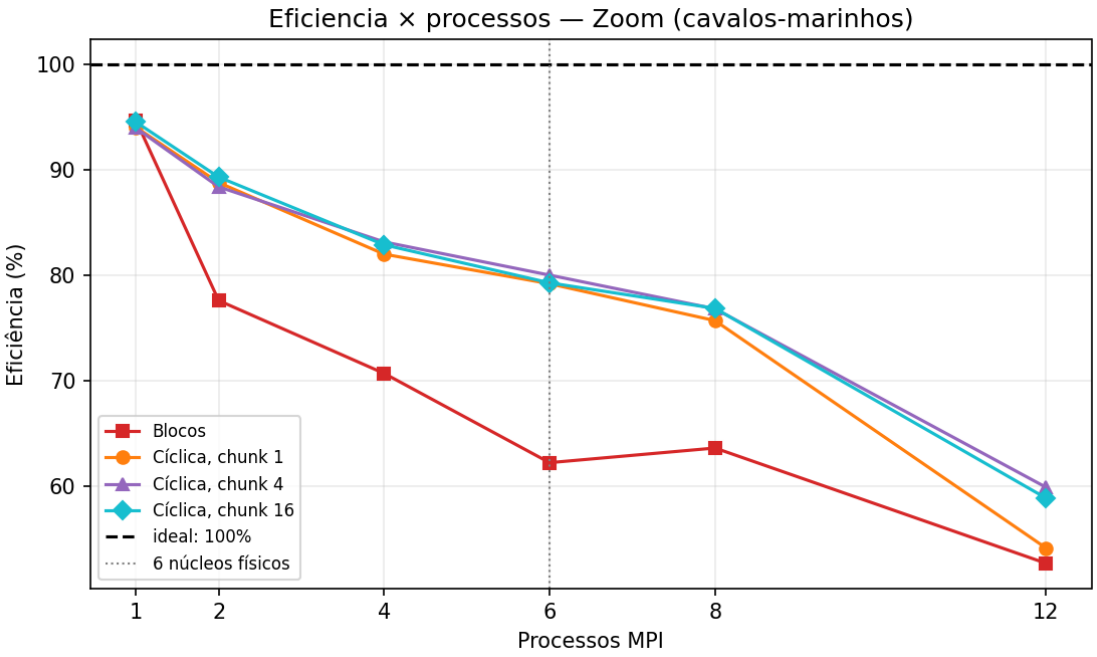


Figura 7: Eficiência paralela no zoom, em função do número de processos.

3.3 Balanceamento de carga e trabalho por processo

O custo por linha da região padrão é concentrado em regiões específicas, tornando inadequada a suposição de que faixas contíguas de tamanho semelhante exigem tempos parecidos. Em 12 processos, o FBC mediano de blocos foi de **0,995** na região padrão; a distribuição cíclica com *chunk* 16 apresentou FBC de **0,073**. No zoom, os valores correspondentes foram **0,643** para blocos e **0,158** para cíclica com *chunk* 4.

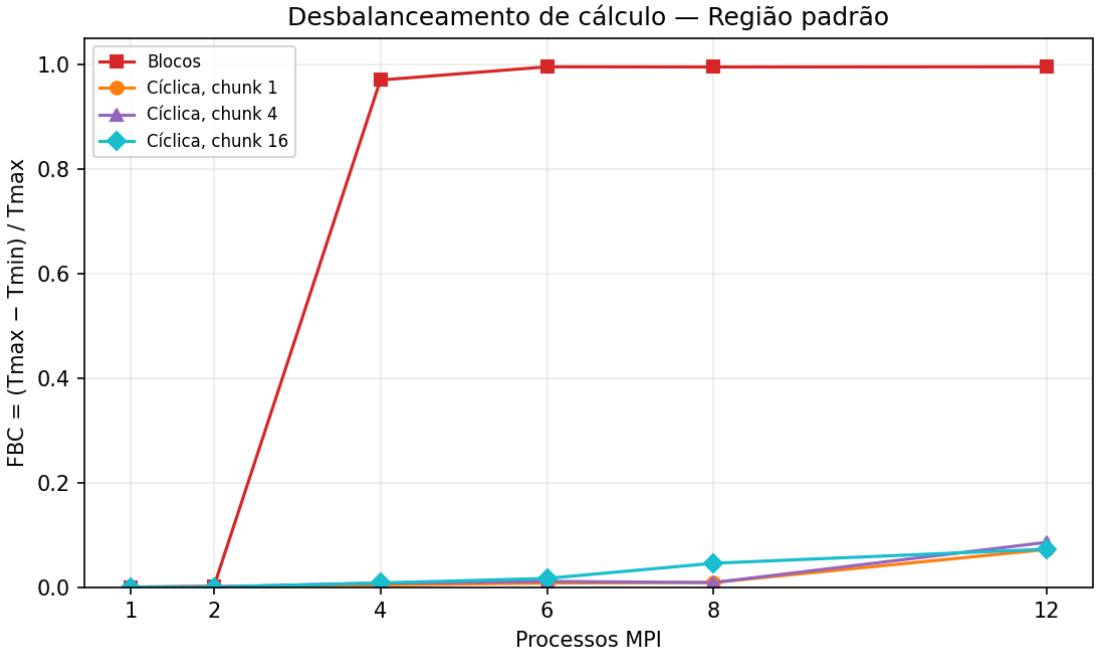


Figura 8: FBC medido a partir dos tempos locais de cálculo: região padrão.

O FBC é calculado em cada execução e depois resumido pela mediana; ele não é obtido tomando o máximo e o mínimo das medianas por rank do gráfico. O desbalanceamento temporal também reflete a execução no sistema operacional: por exemplo, no padrão com $c = 1$ e $p = 12$, o FBC de trabalho (iterações) é apenas 0,000351, mas o FBC de tempo é 0,071867. Os gráficos de custo por linha usam a soma exata das contagens armazenadas como indicador de trabalho, não tempo medido por linha; as três linhas verticais indicam quatro faixas iguais apenas como referência visual.

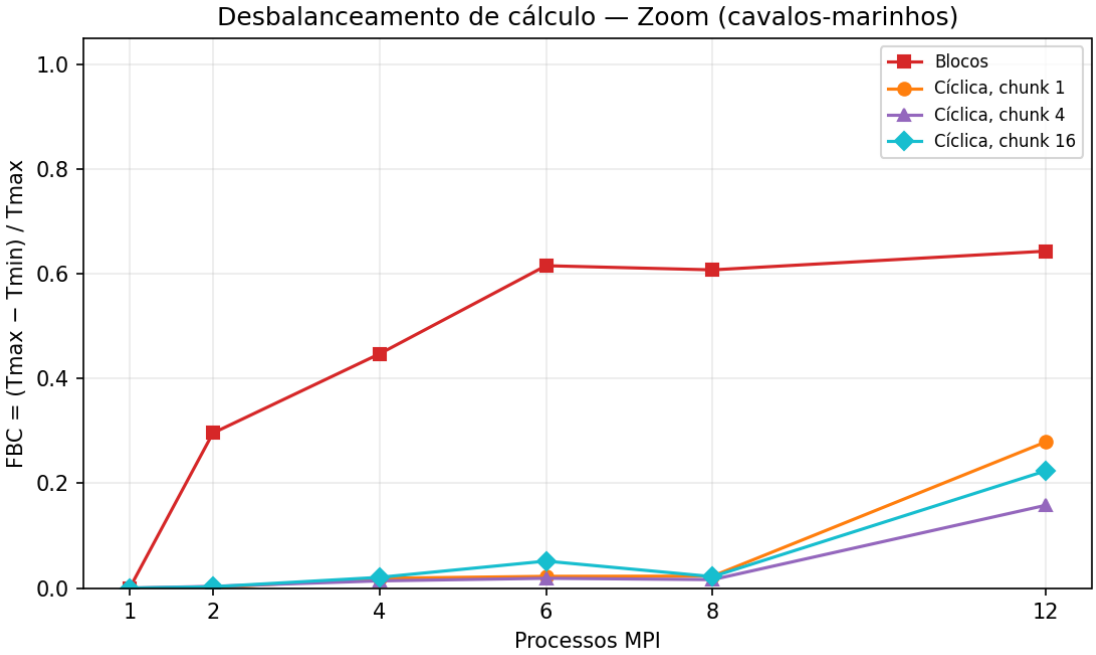


Figura 9: FBC por estratégia no zoom, onde o padrão de trabalho por linha é diferente.

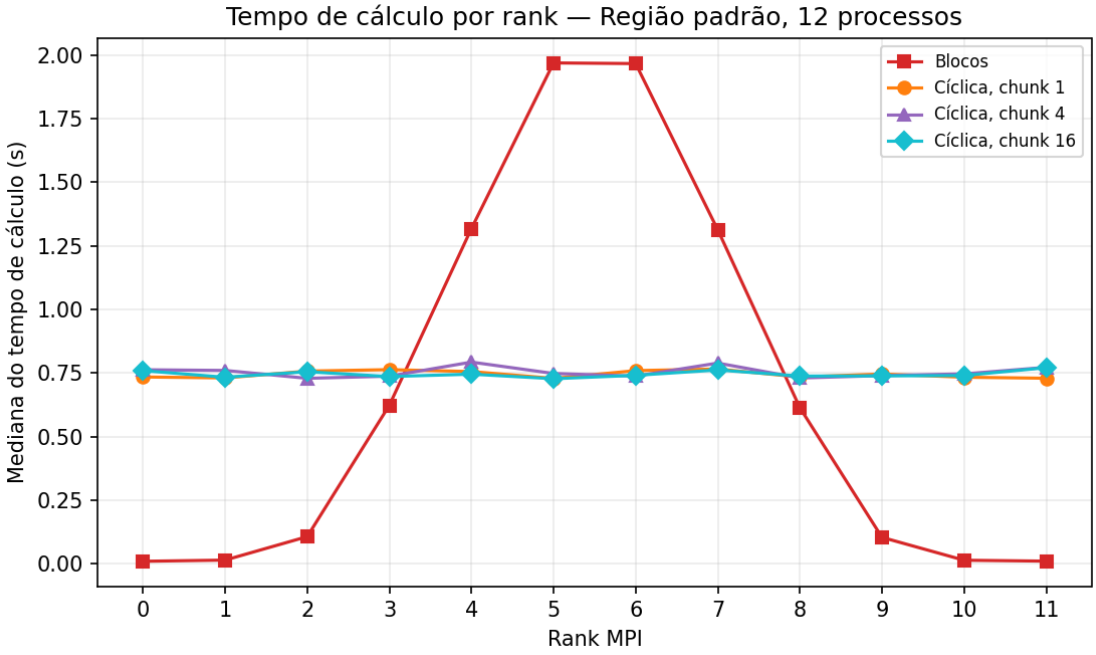


Figura 10: Distribuição do tempo de cálculo entre os 12 ranks no caso padrão.

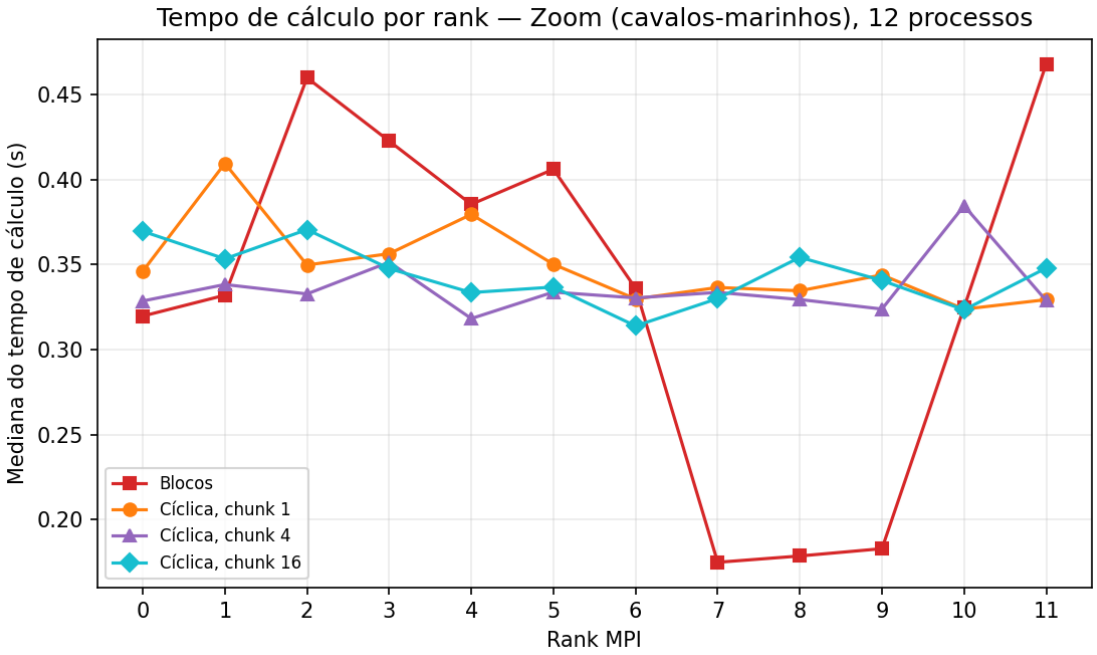


Figura 11: Mediana do tempo de cálculo de cada um dos 12 ranks no zoom, para as quatro configurações; complemento necessário à análise de FBC.

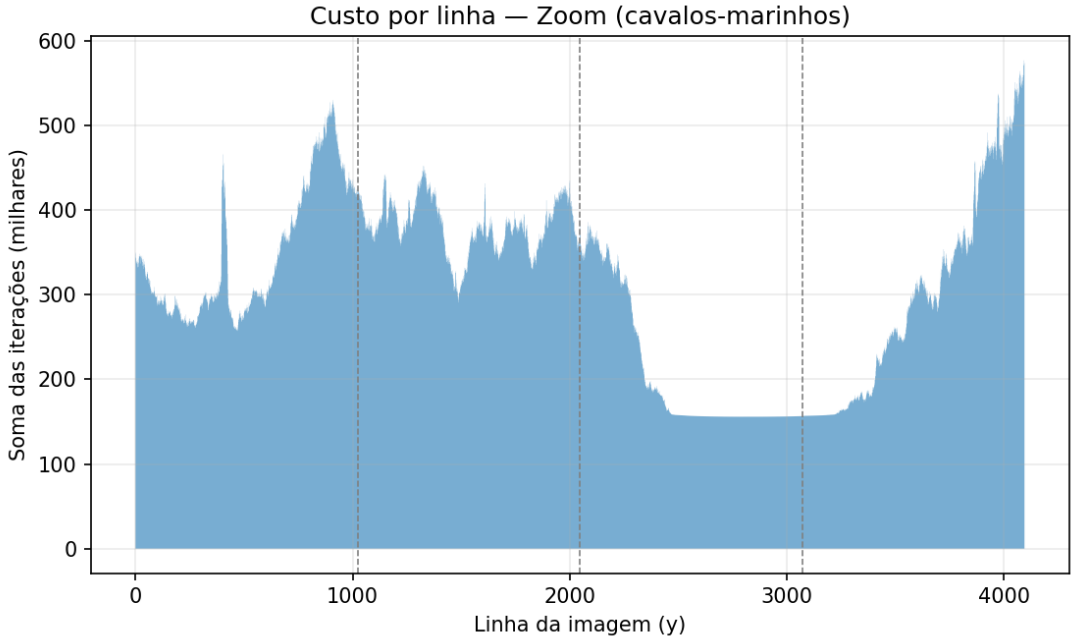


Figura 12: Distribuição do trabalho aproximado por linha no zoom.

3.4 Coleta e comunicação

O custo medido de `MPI_Gatherv` varia conforme a estratégia e a conclusão desigual dos processos. Como não há barreira entre o fim do cálculo e a coleta, os processos mais rápidos podem aguardar dentro da operação coletiva. Assim, o gráfico registra *tempo de coleta observado*, que inclui espera, e não somente transferência de bytes. Na estratégia cíclica, a montagem posterior no raiz é explicitamente contabilizada; nos blocos, a montagem adicional é nula. A escrita do arquivo binário é medida separadamente e excluída dos tempos principais de speedup. As medianas de cálculo máximo e coleta máxima não podem ser somadas para reconstruir o total: os máximos podem pertencer a ranks diferentes e os intervalos se sobrepõem no tempo. No padrão, blocos com $p = 12$ apresenta coleta máxima mediana de 1,979 s para total de 1,988 s; isso evidencia espera, não 99,5% de comunicação pura.

Tabela 4: Componentes medianos em 4096^2, $p = 12$ (s). Cálculo e coleta: máximo entre ranks em cada execução; montagem e escrita: raiz.

Caso	Estratégia	Cálculo	Coleta	Montagem	Escrita
Padrão	Blocos	1,9742	1,9787	0,0000	0,0402
Padrão	Cíclica $c = 1$	0,7754	0,0846	0,0209	0,0439
Padrão	Cíclica $c = 4$	0,8008	0,0827	0,0161	0,0419
Padrão	Cíclica $c = 16$	0,7800	0,0645	0,0152	0,0418
Zoom	Blocos	0,4761	0,3154	0,0000	0,0454
Zoom	Cíclica $c = 1$	0,4317	0,1396	0,0222	0,0476
Zoom	Cíclica $c = 4$	0,3846	0,0866	0,0167	0,0454
Zoom	Cíclica $c = 16$	0,3969	0,1135	0,0150	0,0427

A escrita sequencial mediana foi de 0,042 s no padrão e 0,043 s no zoom. Na escalabilidade fraca com $p = 12$, a escrita MPI variou de 0,557 a 0,585 s entre as medianas das configurações. Esses valores medem a chamada de escrita e fechamento do arquivo, sem exigir persistência física com sincronização explícita do dispositivo.

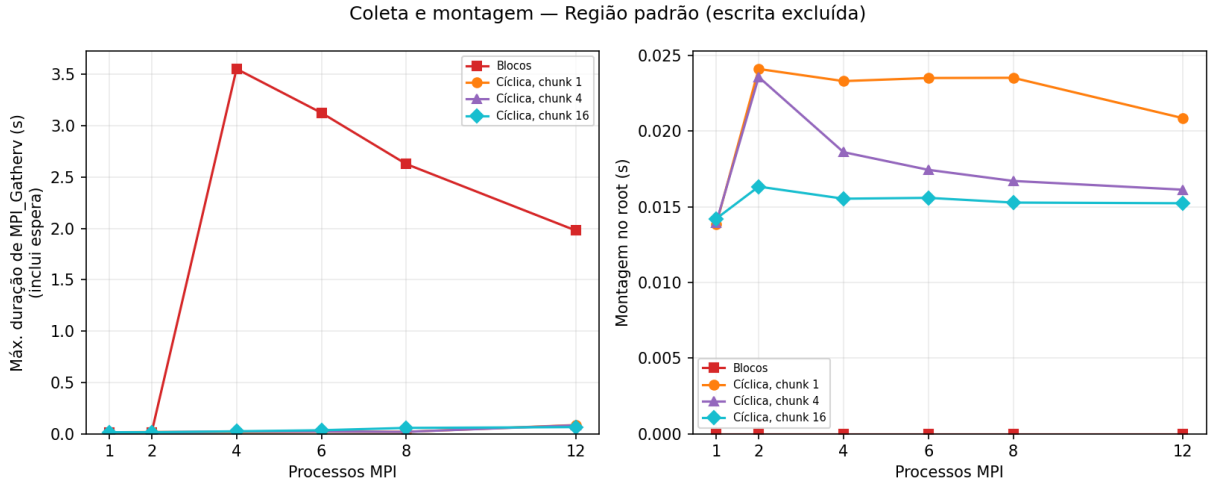


Figura 13: Tempo de `MPI_Gatherv` (com espera) e reorganização no raiz, região padrão.

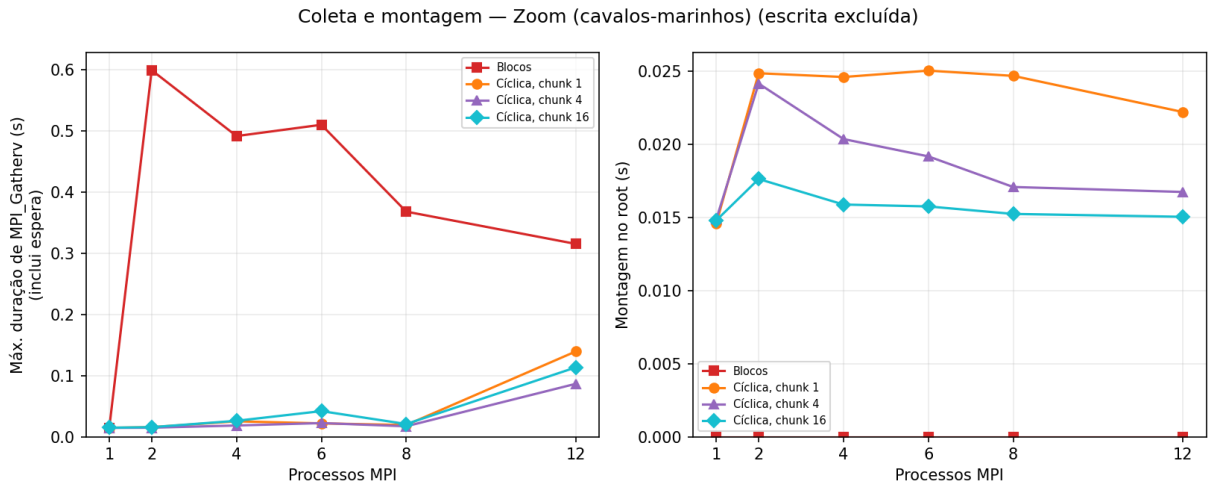


Figura 14: Coleta e reorganização na região de zoom.

3.5 Escalabilidade fraca

A escalabilidade fraca empregou os tamanhos 4096^2, 5793^2, 8192^2, 10033^2, 11585^2 e 14189^2 com 1, 2, 4, 6, 8 e 12 processos. Na maior resolução, o tempo mediano foi de **23,565 s** para blocos e **9,374 s** para cíclica com *chunk* 16. O *baseline* MPI de um processo da mesma estratégia cíclica foi de 7,463 s, resultando em eficiência fraca aproximada de **79,6%**; por blocos, o baseline foi 7,424 s e a eficiência fraca aproximada foi **31,5%**.

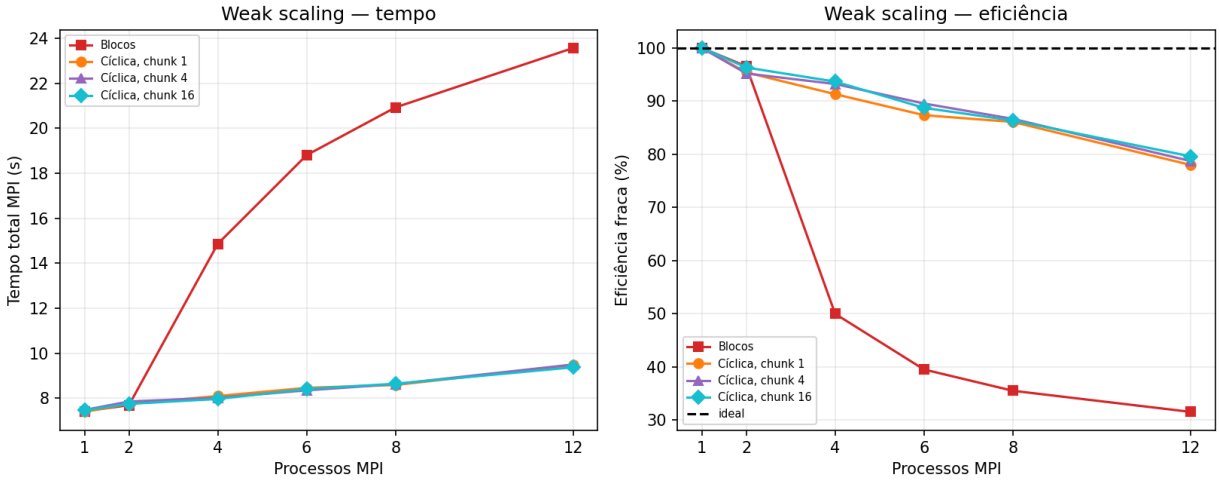


Figura 15: Escalabilidade fraca: tempo total e eficiência fraca por estratégia; trabalho nominal por processo aproximadamente constante.

3.6 Relação com a Etapa 1 (OpenMP)

A Etapa 1 avaliou *threads* OpenMP e políticas `static`, `dynamic` e `guided`; a Etapa 2 avalia processos MPI, faixas contíguas e distribuição cíclica estática. Ambas exploram independência entre pixels. No OpenMP, a matriz é compartilhada e o escalonamento dinâmico entrega trabalho às threads conforme sua disponibilidade; no MPI entregue, a atribuição é estática e há coleta explícita, além

da reorganização no caso cíclico.

A comparação numérica abaixo é **histórica e contextual**. A Etapa 1 registra o mesmo modelo de CPU (i5-12400F), seis núcleos e 32 GB, com Windows build 26200, coleta em 20/09/2026 e GCC 6.3.0 de 32 bits. A Etapa 2 foi coletada em 28/10/2026, com Windows build 26300 e MSVC de 64 bits para MPI. Os baselines OpenMP foram 6,9765 s (padrão) e 3,0970 s (zoom); os MPI usam os baselines sequenciais desta etapa. Portanto, as diferenças abaixo não isolam o custo dos paradigmas em condições controladas.

Tabela 5: Comparação contextual em 4096^2 e 12 threads/processos. Cinco repetições por configuração; speedup e eficiência relativos ao sequencial da respectiva coleta.

Caso	Paradigma	Estratégia	Tempo (s)	Speedup	Eficiência (%)
Padrão	OpenMP	static, blocos	1,871	3,729	31,07
Padrão	OpenMP	static, c=4	0,744	9,377	78,14
Padrão	OpenMP	dynamic, c=1	0,709	9,840	82,00
Padrão	MPI	Blocos	1,988	3,512	29,26
Padrão	MPI	Cíclica, c=4	0,829	8,424	70,20
Zoom	OpenMP	static, blocos	0,443	6,991	58,26
Zoom	OpenMP	static, c=4	0,335	9,245	77,04
Zoom	OpenMP	dynamic, c=1	0,323	9,588	79,90
Zoom	MPI	Blocos	0,489	6,318	52,65
Zoom	MPI	Cíclica, c=4	0,430	7,183	59,86

Nas medições históricas selecionadas, OpenMP apresentou tempos menores. A diferença é compatível com custos distintos de coleta, cópia e compilação, mas não permite atribuição causal exclusiva à comunicação MPI. A comparação de `static,4` com cíclica $c = 4$ aproxima a política de distribuição; `dynamic,1` acrescenta o contraste com balanceamento sob demanda. O MPI permite processos com memórias independentes e execução entre máquinas, possibilidade que não foi avaliada nesta bateria local.

4 Discussão

A consistência interna MPI–sequencial foi verificada nos registros da bateria e nos binários preservados, mas a discrepância em relação ao gerador Python permanece uma **limitação de conformidade externa**; ela não pode ser descrita como aprovação formal. Parte da diferença se explica pela indexação i versus $i + 1$, mas o resíduo após normalização, concentrado em pixels sensíveis da dinâmica iterativa, ultrapassou os limites estabelecidos. Também não é possível atribuir a causa restante a um único fator sem investigação adicional da aritmética de ponto flutuante e das convenções de escape.

A estratégia cíclica reduziu o tempo mediano forte no padrão a partir de quatro processos e no zoom a partir de dois; na escalabilidade fraca, sua vantagem tornou-se expressiva a partir de quatro processos. No padrão forte com dois processos, blocos foi ligeiramente mais rápido: 3,784 s, contra 3,806–3,822 s nas configurações cíclicas. A simetria aproximada da carga em duas metades ajuda a explicar o bom balanceamento desse ponto. Para números maiores de processos, a cíclica espalha regiões com custos diferentes e evita concentrar o trabalho em poucos ranks. Os altos FBC de blocos mostram que a equivalência em número de linhas não garante equivalência em tempo de cálculo. No zoom, a vantagem da distribuição cíclica permanece, porém com diferenças relativas menores em algumas configurações.

O aumento de p reduz o tempo, mas coleta, montagem no raiz e sincronização permanecem na janela medida. A criação dos processos não foi cronometrada nessa janela. A lei de Amdahl, $S(p) = 1/[f + (1 - f)/p]$, explica qualitativamente o limite imposto por uma fração serial f em problema fixo; aqui o custo adicional de comunicação e o desbalanceamento também variam com p, de modo que não se estimou uma fração serial constante. A lei de Gustafson, $S_G(p) = p - \alpha(p - 1)$, motiva a análise com problema crescente, mas a eficiência fraca medida não é uma estimativa direta de S_G. Além da carga iterativa, crescem o volume coletado e a memória no raiz: em 14189^2, cada matriz int32 ocupa cerca de 768 MiB, e a cíclica mantém duas matrizes globais (aproximadamente 1,5 GiB), além do buffer local.

O computador contém seis núcleos físicos e doze processadores lógicos: resultados com 8 e 12 processos devem ser interpretados considerando o uso de *hyper-threading*, compartilhamento de recursos e custos do subsistema de memória. Não foi executada uma investigação isolada de afinidade, NUMA, largura de banda ou comunicação entre máquinas; assim, o experimento caracteriza **MPI local em um único host**, não desempenho de um cluster distribuído.

A estratégia cíclica requer um buffer de recepção adicional e montagem ordenada no raiz; blocos oferece composição mais simples, embora tenha apresentado desbalanceamento severo. Melhorias futuras incluem análise de afinidade, distribuição dinâmica mestre–trabalhador com filas de tarefas, redução de cópias, revisão da convenção de contagem do gerador externo e uma comparação OpenMP–MPI com condições alinhadas.

5 Conclusão

A Etapa 2 implementou e avaliou paralelismo com MPI na geração do conjunto de Mandelbrot, preservando o código sequencial da Etapa 1 e produzindo uma bateria oficial de **386 execuções com consistência interna verificada**. Nos testes internos, as matrizes MPI foram idênticas às sequenciais, enquanto a comparação com o gerador Python do professor identificou discrepâncias preexistentes que exigem documentação explícita.

Na região padrão, a distribuição cíclica com 12 processos atingiu **8,682 vezes** de speedup e **72,35%** de eficiência, superando blocos contíguos. No zoom, a melhor configuração destacada atingiu **7,183 vezes** de speedup. A escalabilidade fraca reforçou o efeito do balanceamento de carga: com 12 processos, a distribuição cíclica manteve aproximadamente **79,6%** de eficiência temporal relativa, contra aproximadamente **31,5%** para blocos. O principal aprendizado foi que, em trabalho irregular, a estratégia de distribuição pode ser tão determinante quanto a quantidade de processos, e que o custo de operações coletivas deve ser separado conceitualmente da espera por desbalanceamento.

Em comparação com o DGEMM estudado em sala, Mandelbrot possui pixels independentes, mas número de iterações irregular. No DGEMM denso, blocos de dimensões semelhantes têm esforço aritmético mais regular e o reaproveitamento de dados/cache é central. Aqui, distribuir o mesmo número de linhas não assegura equilíbrio: a escolha de faixas ou blocos cíclicos altera diretamente o caminho crítico. Em ambos os problemas, sincronização, movimentação de dados e partes seriais limitam os ganhos, mas o desbalanceamento por pixel tem papel particularmente forte no Mandelbrot.

6 Referências

1. UESC. *DEC107 — Processamento Paralelo: aulas de MPI e materiais da disciplina.* Material didático disponibilizado pelo professor, 2026, incluindo o estudo de DGEMM.

2. UESC. *Projeto Conjunto de Mandelbrot — Etapa 2: enunciado e modelo de relatório.* DEC107, 2026.

3. MPI Forum. *MPI: A Message-Passing Interface Standard.* Documentação da API, operações coletivas e temporização. `https://www.mpi-forum.org/docs/`.

4. LUCAS CÉSAR, G. O.; PORTO FILHO, R. R. *Etapa 1 — Benchmark de Geração do Conjunto de Mandelbrot: sequencial e OpenMP.* Relatório da disciplina DEC107, 22 set. 2026.

5. Arquivos do experimento: `resultados_consolidados.csv`, `ambiente.md`, `validacao_referencia_professor.md`, `analise.py` e fontes C, outubro de 2026.

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Nós, **Gabriel Oliveira Lucas César** e **Ronaldo Ribeiro Porto Filho**, regularmente matriculados no curso de Ciência da Computação, declaramos, para os devidos fins, que, na elaboração do trabalho/projeto intitulado "Etapa 2 — Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time): Implementação sequencial e paralela em memória distribuída com MPI", submetido como requisito para a disciplina DEC107 — Processamento Paralelo, adotamos as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

(Assinale a opção aplicável)

( ) Não utilizamos ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

(X) Utilizamos ferramentas de Inteligência Artificial (ex.: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

Detalhamento do Uso (se aplicável): As ferramentas utilizadas por ambos os integrantes foram **ChatGPT e Codex**. O uso se deu exclusivamente nas seguintes etapas do projeto:

- Apoio ao desenvolvimento, explicação, depuração e revisão das implementações em C e MPI e dos scripts auxiliares.
- Planejamento e automação dos testes, consolidação dos resultados, conferência das métricas e geração e análise dos gráficos.
- Auxílio na interpretação de tempo, speedup, eficiência, balanceamento de carga, comunicação e escalabilidade, incluindo a comparação com OpenMP da Etapa 1.
- Investigação e documentação das diferenças numéricas em relação à referência do professor.
- Organização e revisão técnica, gramatical e estrutural do relatório, do README e desta declaração, incluindo a comparação conceitual com DGEMM.

Termo de Responsabilidade: Declaramos estar cientes de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atestamos que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente nossa. Confirmamos que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por nós, garantindo que os resultados refletem nosso próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

A validação crítica inclui a identificação e o registro da divergência perante a referência do professor, descrita neste relatório; não implica aprovação no critério externo de corretude. A bateria oficial foi executada no computador de Gabriel, e suas medições foram preservadas.

Ilhéus – BA, 29 de outubro de 2026.

Gabriel Oliveira Lucas César Matrícula: 202410123	Ronaldo Ribeiro Porto Filho Matrícula: 202410131